SCTR's

PUNE INSTITUTE OF COMPUTER TECHNOLOGY

T. Y. B. TECH
INFORMATION TECHNOLOGY
(2026 COURSE)

LABORATORY MANUAL
FOR

Operating System Laboratory (OSL)

SEMESTER – III

[Subject code: 2635PC31]

[Prepared By]

Ms. Rachana A. Karnavat

Mr. Vishal R. Jaiswal

Mrs. Deepali P. Salapurkar

INSTITUTE VISION AND MISSION

VISION

SCTR's Pune Institute of Computer Technology envisions to be a centre of excellence in higher technical education and research, producing global leaders in the field of technology.

MISSION

- Provide rigorous academic training that equips students to excel in their professional careers with the development of universal human values.
- Foster research and innovation culture amongst faculty and students by encouraging interdisciplinary collaborations, thereby contributing to technological advancements.
- Nurture leadership qualities in students through development of analytical thinking, problem-solving, and effective communication skills.

DEPARTMENT VISION AND MISSION

VISION

The department aspires to be a transformative force in Information Technology education and research, developing globally competitive professionals.

MISSION

- Inculcate an environment of academic rigor in Information Technology education promoting critical thinking, creativity, and problem-solving skills.
- Foster research and industry collaboration by promoting multidisciplinary engagement among students and faculty, driving technological advancements to address societal challenges.

- Cultivate skilled professionals who embody effective communication, leadership, and ethical values.

INDEX

Sr.	Title of Experiment	CO
1	1a) Linux/Unix environment: basic commands, shell scripting, file permissions and process utilities. 1b)Develop a mini student record management system using shell scripts that provides simple database operations (CRUD: Create, Read, Update, Delete) on a plain text file.	CO1
2	2a) Advanced process management: fork() even/odd, orphan & zombie processes, wait() 2b) Parent–child design using fork() and exec() to compute factorial via command-line argument	CO2
3	CPU scheduling algorithms: FCFS, SJF, Priority and Round Robin	CO2
4	IPC: simple chat application using shared memory with synchronization	CO3
5	Synchronization: Producer–Consumer and Reader–Writer problems using semaphores/mutex	CO4
6	Memory allocation: contiguous First Fit, Best Fit and Worst Fit	CO4
7	Page replacement: FIFO and LRU (Optimal as home assignment), with page-fault count	CO5
8	Disk scheduling: FCFS, SSTF, SCAN, C-SCAN with seek-time calculation	CO6
9	CPU scheduling algorithms: FCFS, SJF, Priority and Round Robin	CO6

Course Outcomes (CO1–CO6) are mapped per experiment as per the syllabus.

EXPERIMENT 01

Familiarization with the Linux/Unix Environment

AIM: To familiarize with the Linux/Unix environment — at least 20 basic commands, file/directory navigation, file permissions, process utilities, and to build a shell-script mini project.	**CO1**

THEORY

Unix/Linux is a multi-user, multitasking operating system. The shell (e.g. bash) is the command interpreter that reads typed commands and asks the kernel to run them. Commands operate on a hierarchical file system whose root is "/".

File permissions control who may read (r), write (w) and execute (x) a file. There are three classes: owner, group and others. Permissions are shown by `ls -l` as a 10-character string, e.g. `-rwxr-xr--`, and set numerically with `chmod` (r=4, w=2, x=1).

Process utilities such as `ps`, `top` and `kill` let a user inspect running processes and signal them.

PART A — TWENTY BASIC COMMANDS

Command	Purpose	Example
ls	List directory contents	`ls -l, ls -a`
pwd	Print working directory	`pwd`
cd	Change directory	`cd /home, cd ..`
mkdir	Make a new directory	`mkdir lab1`
rmdir	Remove an empty directory	`rmdir old`
touch	Create an empty file / update time	`touch a.txt`
cp	Copy files or directories	`cp a.txt b.txt`
mv	Move or rename	`mv a.txt docs/`
rm	Remove files	`rm -i a.txt`
cat	Display / concatenate files	`cat file.txt`
head	First lines of a file	`head -5 f.txt`
tail	Last lines of a file	`tail -5 f.txt`
grep	Search text by pattern	`grep "main" a.c`
find	Search for files	`find . -name "*.c"`
diff	Compare two files	`diff a.txt b.txt`
chmod	Change file permissions	`chmod 755 s.sh`

Command	Purpose	Example
chown	Change file owner	`chown user f.txt`
man	Manual page for a command	`man ls`
ps	Report process status	`ps -ef`
top	Live process / resource monitor	`top`
kill	Send a signal to a process	`kill -9 1234`

Permission example: `chmod 754 file` → owner rwx(7), group r-x(5), others r--(4).

PART B — MINI STUDENT RECORD SYSTEM (CRUD ON CSV)

A shell script that performs Create, Read, Update and Delete operations on a plain-text CSV file, demonstrating text parsing with grep, sed and awk and modular function-based scripting.

students.sh

```
#!/bin/bash
# Mini Student Record Management System (CRUD on a CSV file)
DB="students.csv"
[ -f "$DB" ] || echo "Roll,Name,Branch,CGPA" > "$DB"

create() {
    read -p "Enter Roll No: " roll
    if grep -q "^$roll," "$DB"; then
        echo "Record with Roll $roll already exists."; return
    fi
    read -p "Enter Name: " name
    read -p "Enter Branch: " branch
    read -p "Enter CGPA: " cgpa
    echo "$roll,$name,$branch,$cgpa" >> "$DB"
    echo "Record added successfully."
}

read_all() {
    echo "-----------------------------------------------"
    awk -F, '{ printf "%-8s %-18s %-10s %-6s\n", $1, $2, $3, $4 }' "$DB"
    echo "-----------------------------------------------"
}

update() {
    read -p "Enter Roll No to update: " roll
    if ! grep -q "^$roll," "$DB"; then echo "Record not found."; return; fi
    read -p "Enter new Name: " name
    read -p "Enter new Branch: " branch
    read -p "Enter new CGPA: " cgpa
    sed -i "s/^$roll,.*/$roll,$name,$branch,$cgpa/" "$DB"
    echo "Record updated successfully."
}

delete() {
    read -p "Enter Roll No to delete: " roll
    if ! grep -q "^$roll," "$DB"; then echo "Record not found."; return; fi
    sed -i "/^$roll,/d" "$DB"
    echo "Record deleted successfully."
}

while true; do
    echo ""
    echo "==== Student Record Management ===="
    echo "1. Create  2. Read  3. Update  4. Delete  5. Exit"
    read -p "Enter choice: " ch
```

```
    case $ch in
        1) create ;;
        2) read_all ;;
        3) update ;;
        4) delete ;;
        5) echo "Exiting."; exit 0 ;;
        *) echo "Invalid choice." ;;
    esac
done
```

SAMPLE OUTPUT

```
==== Student Record Management ====
1. Create  2. Read  3. Update  4. Delete  5. Exit
Record added successfully.

==== Student Record Management ====
1. Create  2. Read  3. Update  4. Delete  5. Exit
Record added successfully.

==== Student Record Management ====
1. Create  2. Read  3. Update  4. Delete  5. Exit
-------------------------------------------
Roll     Name               Branch     CGPA
101      Amit Sharma        Computer   9.1
102      Neha Patil         IT         8.7
-------------------------------------------

==== Student Record Management ====
1. Create  2. Read  3. Update  4. Delete  5. Exit
Record updated successfully.

==== Student Record Management ====
1. Create  2. Read  3. Update  4. Delete  5. Exit
-------------------------------------------
Roll     Name               Branch     CGPA
101      Amit Sharma        Computer   9.4
102      Neha Patil         IT         8.7
-------------------------------------------

==== Student Record Management ====
1. Create  2. Read  3. Update  4. Delete  5. Exit
Record deleted successfully.

==== Student Record Management ====
1. Create  2. Read  3. Update  4. Delete  5. Exit
-------------------------------------------
Roll     Name               Branch     CGPA
101      Amit Sharma        Computer   9.4
-------------------------------------------

==== Student Record Management ====
1. Create  2. Read  3. Update  4. Delete  5. Exit
Exiting.
```

CONCLUSION

The Linux environment, essential commands, file-permission model and process utilities were studied, and a modular shell script implementing CRUD operations on a CSV file was developed and tested successfully.

EXPERIMENT 02

Process Creation with fork(): Even/Odd, Orphan & Zombie, wait()

AIM: To create a child process using fork() where the parent prints even numbers and the child prints odd numbers, display PIDs/PPIDs, demonstrate orphan and zombie processes, and use wait() to reap the child.	**CO2**

THEORY

`fork()` creates a new process by duplicating the calling process. It returns `0` to the child, the child's PID to the parent, and `-1` on failure. Parent and child then run concurrently from the statement after fork().

Orphan process: if the parent terminates before the child, the child is "adopted" by the `init/systemd` process (PID 1), which becomes its new parent (PPID = 1).

Zombie process: if a child terminates but the parent has not called `wait()`, the child's entry stays in the process table as a "defunct" (zombie) until the parent reaps it. `wait()` makes the parent block until a child finishes and collects its exit status, preventing zombies.

ALGORITHM

1. Call fork() to create a child process.
2. In the child (return value 0): print PID/PPID and all odd numbers.
3. In the parent (return value > 0): print PID/PPID and all even numbers.
4. Parent calls wait(NULL) to reap the child and avoid a zombie.
5. For orphan demo: let the parent exit first; observe child's PPID become 1.
6. For zombie demo: let the child exit while the parent sleeps without wait().

PROGRAM — forkoddeven.c

```
/* Exp 2.1: fork() - parent prints even numbers, child prints odd numbers.
   Shows PIDs/PPIDs; parent uses wait() to reap the child (avoids zombie). */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }
    else if (pid == 0) {                /* Child process */
        printf("CHILD : PID = %d, PPID = %d -> prints ODD numbers\n",
               getpid(), getppid());
        for (int i = 1; i <= 10; i += 2)
            printf("  Child (odd) : %d\n", i);
    }
    else {                              /* Parent process */
        printf("PARENT: PID = %d, PPID = %d -> prints EVEN numbers\n",
               getpid(), getppid());
        for (int i = 2; i <= 10; i += 2)
            printf("  Parent (even): %d\n", i);
```

```
        wait(NULL);                     /* wait for child, reap it */
        printf("PARENT: Child reaped successfully (no zombie left).\n");
    }
    return 0;
}
```

SAMPLE OUTPUT

```
CHILD : PID = 723, PPID = 722 -> prints ODD numbers
  Child (odd) : 1
  Child (odd) : 3
  Child (odd) : 5
  Child (odd) : 7
  Child (odd) : 9
PARENT: PID = 722, PPID = 720 -> prints EVEN numbers
  Parent (even): 2
  Parent (even): 4
  Parent (even): 6
  Parent (even): 8
  Parent (even): 10
PARENT: Child reaped successfully (no zombie left).
```

ORPHAN DEMONSTRATION — orphan.c

```
/* Orphan demo: parent exits first; child is adopted by init/systemd (PPID=1). */
#include <stdio.h>
#include <unistd.h>
int main() {
    pid_t pid = fork();
    if (pid == 0) {
        sleep(2);                                   /* child outlives parent */
        printf("CHILD : now an ORPHAN. PID=%d, new PPID=%d\n",
               getpid(), getppid());
    } else {
        printf("PARENT: PID=%d exiting immediately.\n", getpid());
    }
    return 0;
}
```

Output:

```
PARENT: PID=724 exiting immediately.
CHILD : now an ORPHAN. PID=725, new PPID=1
```

ZOMBIE DEMONSTRATION — zombie.c

```
/* Zombie demo: child exits but parent does NOT wait(), so child stays
   as a zombie (defunct) until the parent ends. */
#include <stdio.h>
#include <unistd.h>
int main() {
    pid_t pid = fork();
    if (pid == 0) {
        printf("CHILD : PID=%d exiting now -> becomes ZOMBIE.\n", getpid());
    } else {
        printf("PARENT: PID=%d sleeping, NOT calling wait().\n", getpid());
        printf("Run 'ps -el | grep defunct' in another terminal to see the zombie.\n");
        sleep(2);
    }
    return 0;
}
```

Output:

```
CHILD : PID=728 exiting now -> becomes ZOMBIE.
PARENT: PID=727 sleeping, NOT calling wait().
Run 'ps -el | grep defunct' in another terminal to see the zombie.
```

CONCLUSION

Process creation using fork() was demonstrated with the parent and child printing even and odd numbers independently. Orphan (PPID becomes 1) and zombie (defunct) conditions were shown, and wait() was used in the parent to reap the child correctly.

EXPERIMENT 03

Process Image Replacement using exec(): Factorial Computation

AIM: To design two separate programs — a parent and a child — where the parent forks a child that uses an exec() function (execvp) to run a factorial program taking the number as a command-line argument, with the parent waiting for completion.	**CO2**

THEORY

The exec() family (execv, execvp, execl, …) replaces the current process image with a new program. The PID stays the same, but the code, data and stack are entirely replaced. Code after a successful exec() never runs.

Typical pattern: `fork()` creates a child; the child calls `exec()` to run a different program; the parent calls `wait()` to synchronise. `execvp` takes the program name and an argument vector and searches the PATH.

ALGORITHM

1. Parent program reads a number N from the command line.
2. Parent calls fork() to create a child process.
3. Child builds an argument vector and calls execvp("./factorial", args) to run the factorial program.
4. factorial.c computes N! from its command-line argument and prints the result.
5. Parent calls wait(&status) and, on WIFEXITED, prints a success confirmation.

PROGRAM 1 — factorial.c (child program)

```
/* child program: computes factorial of an integer given as argv[1] */
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Usage: %s <number>\n", argv[0]);
        return 1;
    }
    int n = atoi(argv[1]);
    unsigned long long fact = 1;
    for (int i = 2; i <= n; i++) fact *= i;
    printf("CHILD : Factorial of %d = %llu  (PID=%d)\n", n, fact, getpid());
    return 0;
}
```

PROGRAM 2 — parent.c (parent program)

```
/* parent program: forks, child replaces itself with ./factorial via execvp,
   parent waits and confirms completion. */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main(int argc, char *argv[]) {
    if (argc < 2) { printf("Usage: %s <number>\n", argv[0]); return 1; }

    pid_t pid = fork();
    if (pid == 0) {                          /* Child */
        char *args[] = { "./factorial", argv[1], NULL };
        execvp(args[0], args);               /* replace image */
        perror("execvp failed");             /* only if exec fails */
```

```
        exit(1);
    } else {                                    /* Parent */
        printf("PARENT: PID=%d created child PID=%d, waiting...\n",
               getpid(), pid);
        int status;
        wait(&status);
        if (WIFEXITED(status))
            printf("PARENT: Child completed successfully (exit code %d).\n",
                   WEXITSTATUS(status));
    }
    return 0;
}
```

COMPILATION & EXECUTION

```
$ gcc factorial.c -o factorial
$ gcc parent.c -o parent
$ ./parent 5
```

SAMPLE OUTPUT

```
CHILD : Factorial of 5 = 120  (PID=730)
PARENT: PID=729 created child PID=730, waiting...
PARENT: Child completed successfully (exit code 0).
```

CONCLUSION

The exec() mechanism was used to replace a child process image with the factorial program passed a command-line argument, while the parent waited for and confirmed the child's successful completion.

EXPERIMENT 03

Implementation and Analysis of CPU Scheduling Algorithms

AIM: To implement and analyse CPU scheduling algorithms — FCFS, SJF (non-preemptive), Priority and Round Robin — and compute the average waiting time and average turnaround time for each.	**CO3**

THEORY

CPU scheduling decides which ready process gets the CPU next. Performance is measured mainly by waiting time (time spent in the ready queue) and turnaround time (submission to completion).

- **FCFS:** processes run in arrival order; simple but suffers the convoy effect.
- **SJF:** the process with the shortest CPU burst runs next; gives optimal average waiting time.
- **Priority:** the highest-priority process runs first (here, a lower number = higher priority); may cause starvation, fixed by aging.
- **Round Robin:** each process gets a fixed time quantum in turn (preemptive); fair and ideal for time-sharing.

ALGORITHM (general)

1. Read the number of processes with their burst times (and priorities for Priority scheduling).
2. For FCFS, keep arrival order; for SJF sort by burst time; for Priority sort by priority.
3. Compute waiting time: WT[0]=0, WT[i]=WT[i-1]+BT[i-1].
4. Compute turnaround time: TAT[i]=WT[i]+BT[i].
5. For Round Robin, cycle through processes subtracting the quantum until each burst is zero.
6. Print each process's WT and TAT and their averages.

PROGRAM — cpu_sched.c

```
/* Exp 3: CPU Scheduling - FCFS, SJF (non-preemptive), Priority, Round Robin.
   Computes waiting time (WT) and turnaround time (TAT) and their averages. */
#include <stdio.h>

void printResult(int n, int p[], int bt[], int wt[], int tat[]) {
    float awt = 0, atat = 0;
    printf("\nPID\tBurst\tWaiting\tTurnaround\n");
    for (int i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\n", p[i], bt[i], wt[i], tat[i]);
        awt += wt[i]; atat += tat[i];
    }
    printf("Average Waiting Time    = %.2f\n", awt / n);
    printf("Average Turnaround Time = %.2f\n", atat / n);
}

void fcfs(int n, int p[], int bt[]) {
    int wt[20], tat[20];
    wt[0] = 0;
    for (int i = 1; i < n; i++) wt[i] = wt[i-1] + bt[i-1];
    for (int i = 0; i < n; i++) tat[i] = wt[i] + bt[i];
    printf("\n----- FCFS Scheduling -----");
    printResult(n, p, bt, wt, tat);
}
```

```
void sjf(int n, int p[], int bt[]) {
    int idx[20], b[20], pr[20];
    for (int i = 0; i < n; i++) { b[i] = bt[i]; pr[i] = p[i]; }
    for (int i = 0; i < n-1; i++)           /* sort by burst time */
      for (int j = 0; j < n-1-i; j++)
        if (b[j] > b[j+1]) {
            int t=b[j]; b[j]=b[j+1]; b[j+1]=t;
            t=pr[j]; pr[j]=pr[j+1]; pr[j+1]=t;
        }
    int wt[20], tat[20];
    wt[0] = 0;
    for (int i = 1; i < n; i++) wt[i] = wt[i-1] + b[i-1];
    for (int i = 0; i < n; i++) tat[i] = wt[i] + b[i];
    printf("\n----- SJF (Non-Preemptive) Scheduling -----");
    printResult(n, pr, b, wt, tat);
}

void priority(int n, int p[], int bt[], int prio[]) {
    int b[20], pr[20], pp[20];
    for (int i = 0; i < n; i++) { b[i]=bt[i]; pr[i]=p[i]; pp[i]=prio[i]; }
    for (int i = 0; i < n-1; i++)           /* sort by priority (low=high) */
      for (int j = 0; j < n-1-i; j++)
        if (pp[j] > pp[j+1]) {
            int t=pp[j]; pp[j]=pp[j+1]; pp[j+1]=t;
            t=b[j]; b[j]=b[j+1]; b[j+1]=t;
            t=pr[j]; pr[j]=pr[j+1]; pr[j+1]=t;
        }
    int wt[20], tat[20];
    wt[0] = 0;
    for (int i = 1; i < n; i++) wt[i] = wt[i-1] + b[i-1];
    for (int i = 0; i < n; i++) tat[i] = wt[i] + b[i];
    printf("\n----- Priority Scheduling (lower number = higher priority) -----");
    printResult(n, pr, b, wt, tat);
}

void roundRobin(int n, int p[], int bt[], int q) {
    int rem[20], wt[20], tat[20], t = 0;
    for (int i = 0; i < n; i++) rem[i] = bt[i];
    int done = 0;
    while (done < n) {
        int didWork = 0;
        for (int i = 0; i < n; i++) {
            if (rem[i] > 0) {
                didWork = 1;
                if (rem[i] > q) { t += q; rem[i] -= q; }
                else { t += rem[i]; wt[i] = t - bt[i]; rem[i] = 0; done++; }
            }
        }
        if (!didWork) break;
    }
    for (int i = 0; i < n; i++) tat[i] = wt[i] + bt[i];
    printf("\n----- Round Robin Scheduling (quantum = %d) -----", q);
    printResult(n, p, bt, wt, tat);
}

int main() {
    int n = 4;
    int p[]    = {1, 2, 3, 4};
    int bt[]   = {6, 8, 7, 3};
    int prio[] = {2, 1, 4, 3};
    int quantum = 3;
    printf("Processes: P1..P4   Burst = {6,8,7,3}   Priority = {2,1,4,3}\n");
    fcfs(n, p, bt);
    sjf(n, p, bt);
    priority(n, p, bt, prio);
    roundRobin(n, p, bt, quantum);
    return 0;
}
```

SAMPLE OUTPUT

```
Processes: P1..P4   Burst = {6,8,7,3}   Priority = {2,1,4,3}

----- FCFS Scheduling -----
PID     Burst   Waiting Turnaround
P1      6       0       6
P2      8       6       14
P3      7       14      21
P4      3       21      24
Average Waiting Time    = 10.25
Average Turnaround Time = 16.25

----- SJF (Non-Preemptive) Scheduling -----
PID     Burst   Waiting Turnaround
P4      3       0       3
P1      6       3       9
P3      7       9       16
P2      8       16      24
Average Waiting Time    = 7.00
Average Turnaround Time = 13.00

----- Priority Scheduling (lower number = higher priority) -----
PID     Burst   Waiting Turnaround
P2      8       0       8
P1      6       8       14
P4      3       14      17
P3      7       17      24
Average Waiting Time    = 9.75
Average Turnaround Time = 15.75

----- Round Robin Scheduling (quantum = 3) -----
PID     Burst   Waiting Turnaround
P1      6       9       15
P2      8       15      23
P3      7       17      24
P4      3       9       12
Average Waiting Time    = 12.50
Average Turnaround Time = 18.50
```

ANALYSIS

For the same workload, SJF gave the lowest average waiting time (7.00), confirming its optimality, while Round Robin gave higher waiting time due to frequent context switches but provides the best responsiveness and fairness.

CONCLUSION

Four CPU scheduling algorithms were implemented and analysed. Their average waiting and turnaround times were compared, demonstrating the trade-offs between efficiency (SJF) and fairness/responsiveness (Round Robin).

EXPERIMENT 04

Inter-Process Communication using Shared Memory (Chat Application)

AIM: To develop a simple chat application using shared memory for communication between two processes, where each process writes to and reads from a shared memory segment with proper synchronization.	**CO4**

THEORY

Shared memory is the fastest IPC mechanism: a memory segment is mapped into the address space of two or more processes, which then read and write it directly. System V calls used are `ftok()` (generate a key), `shmget()` (create/get a segment), `shmat()` (attach), `shmdt()` (detach) and `shmctl()` (control/remove).

Synchronization is needed so the two processes do not access the buffer at the same time. Here a simple `flag` byte acts as a turn indicator: the writer sets flag=1 after writing, and the reader sets flag=0 after reading, so messages are exchanged without conflict.

ALGORITHM

1. Both processes generate the same key with ftok() and obtain the segment with shmget().
2. Each attaches the segment using shmat().
3. Writer waits until flag=0, writes a message, sets flag=1.
4. Reader waits until flag=1, reads and prints the message, sets flag=0.
5. Typing "exit" ends the chat; the reader removes the segment with shmctl(IPC_RMID).

PROGRAM 1 — chat_writer.c

```
/* Exp 4: IPC using System V shared memory - WRITER process.
   Writes messages into a shared segment; reader reads them.
   A status flag byte synchronises turns (0 = reader's turn, 1 = writer's). */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/shm.h>

#define SIZE 1024
struct shm_buf { int flag; char msg[SIZE]; };  /* flag=1 -> new msg from writer */

int main() {
    key_t key = ftok("chatfile", 65);
    int shmid = shmget(key, sizeof(struct shm_buf), 0666 | IPC_CREAT);
    struct shm_buf *buf = (struct shm_buf *) shmat(shmid, NULL, 0);
    buf->flag = 0;

    printf("WRITER ready. Type 'exit' to quit.\n");
    while (1) {
        while (buf->flag == 1) ;            /* wait until reader consumed last */
        printf("Writer> ");
        fgets(buf->msg, SIZE, stdin);
        buf->msg[strcspn(buf->msg, "\n")] = 0;
        buf->flag = 1;                       /* signal reader */
        if (strcmp(buf->msg, "exit") == 0) break;
    }
    shmdt(buf);
    return 0;
```

```
}
```

PROGRAM 2 — chat_reader.c

```
/* Exp 4: IPC using System V shared memory - READER process. */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/shm.h>

#define SIZE 1024
struct shm_buf { int flag; char msg[SIZE]; };

int main() {
    key_t key = ftok("chatfile", 65);
    int shmid = shmget(key, sizeof(struct shm_buf), 0666 | IPC_CREAT);
    struct shm_buf *buf = (struct shm_buf *) shmat(shmid, NULL, 0);

    printf("READER ready.\n");
    while (1) {
        while (buf->flag == 0) ;            /* wait for a new message */
        if (strcmp(buf->msg, "exit") == 0) { printf("Chat ended.\n"); break; }
        printf("Reader received: %s\n", buf->msg);
        buf->flag = 0;                       /* signal writer */
    }
    shmdt(buf);
    shmctl(shmid, IPC_RMID, NULL);           /* remove segment */
    return 0;
}
```

COMPILATION & EXECUTION

```
$ gcc chat_writer.c -o chat_writer
$ gcc chat_reader.c -o chat_reader
# In terminal 1:
$ ./chat_reader
# In terminal 2:
$ ./chat_writer
```

SAMPLE OUTPUT

Writer terminal:

```
WRITER ready. Type 'exit' to quit.
Writer> Writer> Writer>
```

Reader terminal:

```
READER ready.
Reader received: Hello from writer
Reader received: Shared memory works
Chat ended.
```

CONCLUSION

A two-process chat application was implemented using System V shared memory, with a flag-based synchronization scheme ensuring messages are written and read without conflict.

EXPERIMENT 05

Synchronization using Semaphores and Mutex Locks

AIM: To implement the Producer–Consumer and Reader–Writer synchronization problems using POSIX semaphores and mutex locks, ensuring no buffer overflow/underflow and consistent shared data.	**CO4**

THEORY

A semaphore is an integer counter accessed only through `wait (P)` and `signal (V)` operations, used to coordinate access to shared resources. A `mutex` is a binary lock providing mutual exclusion for a critical section.

Producer–Consumer (bounded buffer): the producer adds items, the consumer removes them. Two counting semaphores `empty` and `full` track free and filled slots, and a mutex protects the buffer — preventing overflow and underflow.

Reader–Writer: many readers may read simultaneously, but a writer needs exclusive access. A `readcount` (protected by a mutex) and a `wrt` semaphore ensure the first reader locks out writers and the last reader releases them.

PART A — PRODUCER–CONSUMER

ALGORITHM

1. Initialize empty = buffer size, full = 0, and a mutex.
2. Producer: wait(empty), lock(mutex), add item, unlock(mutex), signal(full).
3. Consumer: wait(full), lock(mutex), remove item, unlock(mutex), signal(empty).

producer_consumer.c

```
/* Exp 5a: Producer-Consumer (Bounded Buffer) using POSIX semaphores + mutex.
   'empty' counts free slots, 'full' counts filled slots, mutex guards buffer. */
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUF 5
#define ITEMS 10
int buffer[BUF], in = 0, out = 0;
sem_t empty, full;
pthread_mutex_t mutex;

void *producer(void *arg) {
    for (int i = 1; i <= ITEMS; i++) {
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);
        buffer[in] = i;
        printf("Produced item %d at slot %d\n", i, in);
        in = (in + 1) % BUF;
        pthread_mutex_unlock(&mutex);
        sem_post(&full);
        usleep(50000);
    }
    return NULL;
}
```

```
void *consumer(void *arg) {
    for (int i = 1; i <= ITEMS; i++) {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);
        int item = buffer[out];
        printf("            Consumed item %d from slot %d\n", item, out);
        out = (out + 1) % BUF;
        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
        usleep(90000);
    }
    return NULL;
}

int main() {
    pthread_t pt, ct;
    sem_init(&empty, 0, BUF);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);
    pthread_create(&pt, NULL, producer, NULL);
    pthread_create(&ct, NULL, consumer, NULL);
    pthread_join(pt, NULL);
    pthread_join(ct, NULL);
    sem_destroy(&empty); sem_destroy(&full);
    pthread_mutex_destroy(&mutex);
    printf("Producer-Consumer finished without overflow/underflow.\n");
    return 0;
}
```

Sample Output:

```
Produced item 1 at slot 0
            Consumed item 1 from slot 0
Produced item 2 at slot 1
            Consumed item 2 from slot 1
Produced item 3 at slot 2
Produced item 4 at slot 3
            Consumed item 3 from slot 2
Produced item 5 at slot 4
Produced item 6 at slot 0
            Consumed item 4 from slot 3
Produced item 7 at slot 1
Produced item 8 at slot 2
            Consumed item 5 from slot 4
Produced item 9 at slot 3
            Consumed item 6 from slot 0
Produced item 10 at slot 4
            Consumed item 7 from slot 1
            Consumed item 8 from slot 2
            Consumed item 9 from slot 3
            Consumed item 10 from slot 4
Producer-Consumer finished without overflow/underflow.
```

PART B — READER–WRITER

reader_writer.c

```
/* Exp 5b: Reader-Writer problem (readers-preference) using mutex + semaphore.
   Many readers may read together; a writer needs exclusive access. */
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

sem_t wrt;                  /* gives a writer exclusive access */
pthread_mutex_t rcmutex;    /* protects readcount */
int readcount = 0;
int shared_data = 0;
```

```
void *reader(void *arg) {
    int id = *(int *)arg;
    pthread_mutex_lock(&rcmutex);
    readcount++;
    if (readcount == 1) sem_wait(&wrt);     /* first reader locks writers out */
    pthread_mutex_unlock(&rcmutex);

    printf("Reader %d reads data = %d  (active readers = %d)\n",
           id, shared_data, readcount);
    usleep(50000);

    pthread_mutex_lock(&rcmutex);
    readcount--;
    if (readcount == 0) sem_post(&wrt);     /* last reader frees writers */
    pthread_mutex_unlock(&rcmutex);
    return NULL;
}

void *writer(void *arg) {
    int id = *(int *)arg;
    sem_wait(&wrt);                          /* exclusive access */
    shared_data += 10;
    printf("Writer %d WROTE data = %d  (exclusive)\n", id, shared_data);
    usleep(50000);
    sem_post(&wrt);
    return NULL;
}

int main() {
    pthread_t r[3], w[2];
    int id[5] = {1, 2, 3, 1, 2};
    sem_init(&wrt, 0, 1);
    pthread_mutex_init(&rcmutex, NULL);

    pthread_create(&r[0], NULL, reader, &id[0]);
    pthread_create(&w[0], NULL, writer, &id[3]);
    pthread_create(&r[1], NULL, reader, &id[1]);
    pthread_create(&r[2], NULL, reader, &id[2]);
    pthread_create(&w[1], NULL, writer, &id[4]);

    for (int i = 0; i < 3; i++) pthread_join(r[i], NULL);
    for (int i = 0; i < 2; i++) pthread_join(w[i], NULL);
    printf("Reader-Writer finished; data consistency maintained.\n");
    return 0;
}
```

Sample Output:

```
Reader 1 reads data = 0  (active readers = 1)
Reader 2 reads data = 0  (active readers = 2)
Reader 3 reads data = 0  (active readers = 3)
Writer 1 WROTE data = 10  (exclusive)
Writer 2 WROTE data = 20  (exclusive)
Reader-Writer finished; data consistency maintained.
```

CONCLUSION

The Producer–Consumer and Reader–Writer problems were solved using POSIX semaphores and mutex locks. Synchronization prevented buffer overflow/underflow and guaranteed data consistency between concurrent threads.

EXPERIMENT 06

Simulation of Memory Allocation Strategies

AIM: To simulate contiguous memory allocation strategies — First Fit, Best Fit and Worst Fit — and observe how each places processes into available memory blocks.	**CO5**

THEORY

In contiguous allocation each process occupies a single block of memory. When several free blocks (holes) exist, a placement strategy decides which hole to use:

- **First Fit:** allocate the first hole large enough. Fast, but leaves fragments near the start.
- **Best Fit:** allocate the smallest hole that fits. Minimises wasted space but creates many tiny holes.
- **Worst Fit:** allocate the largest hole. Leaves large usable leftovers but can exhaust big blocks quickly.

ALGORITHM

1. Read the sizes of memory blocks and processes.
2. For each process, scan the blocks per the chosen strategy (first / smallest / largest suitable).
3. If a suitable block is found, allocate it and reduce that block's free size.
4. Otherwise mark the process as "Not Allocated".
5. Display the block assigned to each process.

PROGRAM — mem_alloc.c

```
/* Exp 6: Contiguous memory allocation - First Fit, Best Fit, Worst Fit. */
#include <stdio.h>

void firstFit(int b[], int nb, int p[], int np) {
    int block[20], alloc[20];
    for (int i = 0; i < nb; i++) block[i] = b[i];
    for (int i = 0; i < np; i++) alloc[i] = -1;
    for (int i = 0; i < np; i++)
        for (int j = 0; j < nb; j++)
            if (block[j] >= p[i]) { alloc[i] = j; block[j] -= p[i]; break; }
    printf("\n--- First Fit ---\nProcess\tSize\tBlock\n");
    for (int i = 0; i < np; i++)
        printf("P%d\t%d\t%s\n", i+1, p[i],
               alloc[i] != -1 ? (char[]){'B', (char)('1'+alloc[i]), 0} : "Not Allocated");
}

void bestFit(int b[], int nb, int p[], int np) {
    int block[20], alloc[20];
    for (int i = 0; i < nb; i++) block[i] = b[i];
    for (int i = 0; i < np; i++) alloc[i] = -1;
    for (int i = 0; i < np; i++) {
        int best = -1;
        for (int j = 0; j < nb; j++)
            if (block[j] >= p[i] && (best == -1 || block[j] < block[best])) best = j;
        if (best != -1) { alloc[i] = best; block[best] -= p[i]; }
    }
    printf("\n--- Best Fit ---\nProcess\tSize\tBlock\n");
    for (int i = 0; i < np; i++)
        printf("P%d\t%d\t%s\n", i+1, p[i],
               alloc[i] != -1 ? (char[]){'B', (char)('1'+alloc[i]), 0} : "Not Allocated");
}
```

```
void worstFit(int b[], int nb, int p[], int np) {
    int block[20], alloc[20];
    for (int i = 0; i < nb; i++) block[i] = b[i];
    for (int i = 0; i < np; i++) alloc[i] = -1;
    for (int i = 0; i < np; i++) {
        int worst = -1;
        for (int j = 0; j < nb; j++)
            if (block[j] >= p[i] && (worst == -1 || block[j] > block[worst])) worst = j;
        if (worst != -1) { alloc[i] = worst; block[worst] -= p[i]; }
    }
    printf("\n--- Worst Fit ---\nProcess\tSize\tBlock\n");
    for (int i = 0; i < np; i++)
        printf("P%d\t%d\t%s\n", i+1, p[i],
               alloc[i] != -1 ? (char[]){'B', (char)('1'+alloc[i]), 0} : "Not Allocated");
}

int main() {
    int blocks[] = {100, 500, 200, 300, 600};
    int procs[]  = {212, 417, 112, 426};
    int nb = 5, np = 4;
    printf("Blocks: 100 500 200 300 600\nProcesses: 212 417 112 426\n");
    firstFit(blocks, nb, procs, np);
    bestFit(blocks, nb, procs, np);
    worstFit(blocks, nb, procs, np);
    return 0;
}
```

SAMPLE OUTPUT

```
Blocks: 100 500 200 300 600
Processes: 212 417 112 426

--- First Fit ---
Process Size    Block
P1      212     B2
P2      417     B5
P3      112     B2
P4      426     Not Allocated

--- Best Fit ---
Process Size    Block
P1      212     B4
P2      417     B2
P3      112     B3
P4      426     B5

--- Worst Fit ---
Process Size    Block
P1      212     B5
P2      417     B2
P3      112     B5
P4      426     Not Allocated
```

CONCLUSION

First Fit, Best Fit and Worst Fit allocation strategies were simulated. The results illustrate how the choice of strategy affects which processes can be accommodated and the resulting fragmentation.

EXPERIMENT 07

Page Replacement Algorithms: FIFO, LRU and Optimal

AIM: To implement FIFO and LRU page replacement algorithms (and Optimal as a home assignment), track page faults for a given reference string, show the frames after each reference, and report the total page faults.	**CO6**

THEORY

In demand paging, a page fault occurs when a referenced page is not in a memory frame. When all frames are full, a page replacement algorithm chooses a victim page to evict.

- **FIFO:** evict the page that has been in memory the longest. Simple, but can suffer Belady's anomaly.
- **LRU:** evict the page that has not been used for the longest time. Good approximation of optimal, needs usage tracking.
- **Optimal:** evict the page that will not be used for the longest time in the future. Gives the minimum faults but needs future knowledge (theoretical benchmark).

ALGORITHM (general)

1. Read the page reference string and the number of frames.
2. For each reference, check whether the page is already in a frame (hit) or not (fault).
3. On a fault with free frames, load the page; otherwise select a victim by the policy (oldest / least-recently-used / farthest-future-use).
4. Display the frame contents after each reference and count faults.
5. Report the total number of page faults.

PROGRAM — page_replace.c

```
/* Exp 7: Page Replacement - FIFO, LRU, and Optimal.
   Tracks page faults for a reference string and given number of frames. */
#include <stdio.h>
#include <limits.h>

int ref[] = {7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1};
int RS, FR = 3;

void show(int f[], int n) {
    printf("  Frames: ");
    for (int i = 0; i < n; i++)
        if (f[i] == -1) printf("- "); else printf("%d ", f[i]);
    printf("\n");
}

void fifo() {
    int f[10], n = FR, ptr = 0, faults = 0;
    for (int i = 0; i < n; i++) f[i] = -1;
    printf("\n===== FIFO =====\n");
    for (int i = 0; i < RS; i++) {
        int hit = 0;
        for (int j = 0; j < n; j++) if (f[j] == ref[i]) hit = 1;
        if (!hit) { f[ptr] = ref[i]; ptr = (ptr+1)%n; faults++;
            printf("Ref %d -> FAULT", ref[i]); }
        else printf("Ref %d -> hit  ", ref[i]);
```

```
        show(f, n);
    }
    printf("Total Page Faults (FIFO) = %d\n", faults);
}

void lru() {
    int f[10], last[10], n = FR, faults = 0;
    for (int i = 0; i < n; i++) { f[i] = -1; last[i] = -1; }
    printf("\n===== LRU =====\n");
    for (int i = 0; i < RS; i++) {
        int hit = -1;
        for (int j = 0; j < n; j++) if (f[j] == ref[i]) hit = j;
        if (hit == -1) {
            int lru = 0;
            for (int j = 1; j < n; j++) if (last[j] < last[lru]) lru = j;
            f[lru] = ref[i]; last[lru] = i; faults++;
            printf("Ref %d -> FAULT", ref[i]);
        } else { last[hit] = i; printf("Ref %d -> hit  ", ref[i]); }
        show(f, n);
    }
    printf("Total Page Faults (LRU) = %d\n", faults);
}

void optimal() {
    int f[10], n = FR, faults = 0;
    for (int i = 0; i < n; i++) f[i] = -1;
    printf("\n===== Optimal =====\n");
    for (int i = 0; i < RS; i++) {
        int hit = 0;
        for (int j = 0; j < n; j++) if (f[j] == ref[i]) hit = 1;
        if (!hit) {
            int idx = -1, farthest = -1;
            for (int j = 0; j < n; j++) {
                if (f[j] == -1) { idx = j; break; }
                int k;
                for (k = i+1; k < RS; k++) if (ref[k] == f[j]) break;
                if (k > farthest) { farthest = k; idx = j; }
            }
            f[idx] = ref[i]; faults++;
            printf("Ref %d -> FAULT", ref[i]);
        } else printf("Ref %d -> hit  ", ref[i]);
        show(f, n);
    }
    printf("Total Page Faults (Optimal) = %d\n", faults);
}

int main() {
    RS = sizeof(ref)/sizeof(ref[0]);
    printf("Reference string: ");
    for (int i = 0; i < RS; i++) printf("%d ", ref[i]);
    printf("\nNumber of frames = %d\n", FR);
    fifo(); lru(); optimal();
    return 0;
}
```

SAMPLE OUTPUT (summary)

```
Reference string: 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1
Number of frames = 3

Total Page Faults (FIFO)    = 15
Total Page Faults (LRU)     = 12
Total Page Faults (Optimal) = 9
```

(Full per-reference trace is produced by the program; see console output.)

HOME ASSIGNMENT

The Optimal page replacement algorithm is implemented in the same program (function optimal()). For this reference string with 3 frames it gives 9 page faults — fewer than LRU (12) and FIFO (15) — confirming it as the theoretical lower bound.

CONCLUSION

FIFO, LRU and Optimal page replacement algorithms were implemented and compared on a common reference string. Optimal produced the fewest faults, LRU was close behind, and FIFO produced the most — matching theoretical expectations.

EXPERIMENT 08

Disk Scheduling Algorithms: FCFS, SSTF, SCAN and C-SCAN

AIM: To implement disk scheduling algorithms — FCFS, SSTF, SCAN and C-SCAN — and calculate the total head movement (seek time) for a given request queue and initial head position.	**CO6**

THEORY

Disk scheduling orders pending disk I/O requests to minimise the total seek time (head movement). The main algorithms are:

- **FCFS:** serve requests in arrival order. Fair but often inefficient.
- **SSTF:** serve the request closest to the current head position next. Reduces seek time but can starve far requests.
- **SCAN (elevator):** the head moves in one direction servicing requests until the end, then reverses.
- **C-SCAN:** like SCAN but after reaching one end the head jumps back to the start and scans again in the same direction, giving more uniform wait times.

ALGORITHM (general)

1. Read the request queue, initial head position and disk size.
2. FCFS: add |request − head| for each request in order.
3. SSTF: repeatedly pick the nearest unserved request to the current head.
4. SCAN: sort requests; move towards one end servicing them, reach the end, then reverse.
5. C-SCAN: move to the end, jump to 0, continue in the same direction.
6. Accumulate and print the total seek time for each.

PROGRAM — disk_sched.c

```
/* Exp 8: Disk Scheduling - FCFS, SSTF, SCAN, C-SCAN with total seek time. */
#include <stdio.h>
#include <stdlib.h>

#define N 8
int abs_i(int x){ return x < 0 ? -x : x; }

void fcfs(int req[], int head) {
    int total = 0, cur = head;
    printf("\n--- FCFS ---\nSeek order: %d", cur);
    for (int i = 0; i < N; i++) { total += abs_i(req[i]-cur); cur = req[i]; printf(" -> %d", cur); }
    printf("\nTotal seek time = %d\n", total);
}

void sstf(int req[], int head) {
    int r[N], done[N] = {0}, cur = head, total = 0;
    for (int i = 0; i < N; i++) r[i] = req[i];
    printf("\n--- SSTF ---\nSeek order: %d", cur);
    for (int c = 0; c < N; c++) {
        int idx = -1, best = 1e9;
        for (int i = 0; i < N; i++)
            if (!done[i] && abs_i(r[i]-cur) < best) { best = abs_i(r[i]-cur); idx = i; }
```

```
        done[idx] = 1; total += best; cur = r[idx]; printf(" -> %d", cur);
    }
    printf("\nTotal seek time = %d\n", total);
}

void scan(int req[], int head, int disk) {
    int r[N+2], n = 0, cur = head, total = 0;
    for (int i = 0; i < N; i++) r[n++] = req[i];
    r[n++] = 0; r[n++] = disk-1;                /* go to both disk ends */
    for (int i = 0; i < n-1; i++)               /* sort ascending */
      for (int j = 0; j < n-1-i; j++)
        if (r[j] > r[j+1]) { int t=r[j]; r[j]=r[j+1]; r[j+1]=t; }
    int pos = 0; while (r[pos] < head) pos++;
    printf("\n--- SCAN (moving towards larger end) ---\nSeek order: %d", cur);
    for (int i = pos; i < n; i++) { if (r[i]==head) continue; total += abs_i(r[i]-cur); cur = r[i];
printf(" -> %d", cur); }
    for (int i = pos-1; i >= 0; i--) { total += abs_i(r[i]-cur); cur = r[i]; printf(" -> %d", cur); }
    printf("\nTotal seek time = %d\n", total);
}

void cscan(int req[], int head, int disk) {
    int r[N+2], n = 0, cur = head, total = 0;
    for (int i = 0; i < N; i++) r[n++] = req[i];
    r[n++] = 0; r[n++] = disk-1;
    for (int i = 0; i < n-1; i++)
      for (int j = 0; j < n-1-i; j++)
        if (r[j] > r[j+1]) { int t=r[j]; r[j]=r[j+1]; r[j+1]=t; }
    int pos = 0; while (r[pos] < head) pos++;
    printf("\n--- C-SCAN ---\nSeek order: %d", cur);
    for (int i = pos; i < n; i++) { if (r[i]==head) continue; total += abs_i(r[i]-cur); cur = r[i];
printf(" -> %d", cur); }
    total += abs_i(cur-0); cur = 0; printf(" -> 0");      /* jump to start */
    for (int i = 0; i < pos; i++) { if (r[i]==cur) continue; total += abs_i(r[i]-cur); cur = r[i];
printf(" -> %d", cur); }
    printf("\nTotal seek time = %d\n", total);
}

int main() {
    int req[N] = {98, 183, 37, 122, 14, 124, 65, 67};
    int head = 53, disk = 200;
    printf("Requests: 98 183 37 122 14 124 65 67\nHead start = %d, Disk size = %d\n", head, disk);
    fcfs(req, head);
    sstf(req, head);
    scan(req, head, disk);
    cscan(req, head, disk);
    return 0;
}
```

SAMPLE OUTPUT

```
Requests: 98 183 37 122 14 124 65 67
Head start = 53, Disk size = 200


--- FCFS ---
Seek order: 53 -> 98 -> 183 -> 37 -> 122 -> 14 -> 124 -> 65 -> 67
Total seek time = 640


--- SSTF ---
Seek order: 53 -> 65 -> 67 -> 37 -> 14 -> 98 -> 122 -> 124 -> 183
Total seek time = 236


--- SCAN (moving towards larger end) ---
Seek order: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 199 -> 37 -> 14 -> 0
Total seek time = 345


--- C-SCAN ---
Seek order: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 199 -> 0 -> 14 -> 37
Total seek time = 382
```

CONCLUSION

FCFS, SSTF, SCAN and C-SCAN disk scheduling algorithms were implemented and their total seek times compared. SSTF and SCAN reduced head movement substantially compared with FCFS, demonstrating the value of intelligent disk scheduling.